Twenty-Seventh Implementation Iteration: Prepare Allocation Model and Strategy Interface

for Future Multi-Room Exam Allocation

Software Design / Iteration Report

Name: Najib Attar
Date: May 2026

1. Iteration Goal

The goal of this twenty-seventh implementation iteration is to prepare the allocation model and allocation strategy interface for future multi-room exam allocation.

Before this iteration, an `Allocation` represented that a request used a specific space at a specific time slot. However, it did not explicitly store how many participants were assigned to that particular space. This was acceptable while every approved request produced one full-room allocation, but it would not be enough for future exam splitting.

This iteration adds `assignedParticipants` to `Allocation`. It also updates the allocation strategy interface so batch strategies can receive the full space pool. These changes prepare the architecture for later exam allocation work, while preserving the current behavior of Greedy and Priority strategies.

2. Changes from Iteration 26

Compared with the twenty-sixth iteration, the following changes were introduced:

- Added `assignedParticipants` to the `Allocation` model
- Added `getAssignedParticipants()` to `Allocation`
- Updated all allocation creation sites to provide assigned participant counts
- Updated `GreedyAllocationStrategy` and `PriorityAllocationStrategy` so approved allocations store the request participant count
- Updated recurring request allocation creation so each occurrence stores the request participant count

- Updated exam request allocation creation so it stores the request participant count for now
- Updated the manual existing allocation used for conflict testing with a realistic assigned participant count
- Updated `AllocationService` console output to show assigned participants
- Updated `AllocationWriter` so `allocations.csv` exports assigned participants
- Updated the strategy batch processing interface so strategies receive the full space list
- Updated `AllocationService` and `main.cpp` to pass loaded spaces into the selected strategy

This iteration does not implement multi-room splitting, best-fit selection, sharing-aware allocation, or capacity-aware availability.

3. Scenario

The prototype still loads users, spaces, requests, and configuration data through the existing data loading workflow. After the data is loaded, `main.cpp` creates an `AllocationService` using the configured allocation strategy. The service now receives both the loaded requests and the loaded spaces when batch processing begins.

The selected strategy still processes requests exactly as before. It does not choose alternate rooms yet, and it does not split exam requests across multiple rooms. However, the full space pool is now available to the strategy interface for future allocation algorithms.

When a request is approved, the created `Allocation` now records the number of participants assigned to that allocation. For current one-time, recurring, and exam requests, this value is equal to the request participant count. At the end of execution, `allocations.csv` includes this value, and console allocation output displays it.

4. Modules and Their Tasks

A. Allocation Model Module

- Continue storing allocation identity, request identity, space, time slot, and status
- Add room-level assigned participant information
- Represent how many participants are assigned to a specific allocation

B. Allocation Strategy Module

- Continue processing requests through Greedy or Priority strategy behavior
- Accept the full loaded space pool during batch processing
- Preserve current allocation behavior while preparing for future strategy logic

C. AllocationService Module

- Continue coordinating selected strategy execution
- Pass loaded requests and loaded spaces to the selected strategy
- Print assigned participant counts in allocation output

D. Writer Module

- Continue exporting approved allocations to `allocations.csv`
- Add `assignedParticipants` to the allocation export
- Preserve all existing useful allocation columns

E. Main Program Flow

- Continue loading data through `DataController`
- Pass both `data.requests` and `data.spaces` into `AllocationService`
- Preserve current request processing and export workflow

5. Assigned Participants and Space Pool Design

The main design idea in this iteration is to distinguish total request demand from room-level assignment.

`Request::participantCount` represents the total number of participants requested. In contrast, `Allocation::assignedParticipants` represents how many of those participants are assigned to one specific room allocation.

For current requests, the values are the same because the system still creates one allocation per one-time or exam request, and one allocation per recurring occurrence. For example, an approved exam request with 35 participants creates one allocation with `assignedParticipants = 35`.

In a future multi-room exam allocation iteration, one exam request may create several allocations. For example:

- BLG101 exam has 120 students
- Allocation 1 assigns 60 students to B201
- Allocation 2 assigns 40 students to B202
- Allocation 3 assigns 20 students to B203

The second preparation step is the updated strategy interface. Strategies now receive the full space pool during batch processing. This means future strategies will have enough information to choose one or more rooms from the loaded spaces. Current strategies intentionally ignore the space pool for now and continue using each request's selected space.

6. Iteration Comparison

Iter.	Focus	Added / Modified	Design Responsibility	Prototype Effect
23	Request creation	`RequestFactory`	Separated request construction from `DataController`.	The controller became cleaner and less responsible for object creation.
24	Request metadata	`title` and `purpose` in `Request`	Extended requests with descriptive information for future use cases.	Requests became clearer and better prepared for exam-oriented extensions.
25	Explicit exam requests	`ExamRequest` and exam-related fields	Separated exam-specific meaning from generic one-time requests.	Exams became first-class requests while still using one-time allocation behavior.
26	Request-type permission	`RequestTypeRule`, `canSubmitRequestType()`, and `getRequestType()`	Separated request submission permission from space-type authorization.	Students and staff can be rejected for submitting exam requests before other rule checks continue.
27	Allocation preparation	`assignedParticipants` and strategy access to spaces	Separated total request demand from room-level assignment and prepared strategies to see the full space pool.	Allocations now record assigned participant counts, and strategies are ready for future multi-room exam allocation logic.

7. Updated Class Diagram

Figure 1 shows the updated class diagram. The main changes are the new `assignedParticipants` field in `Allocation`, the new getter `getAssignedParticipants()`, and the updated strategy batch processing methods that receive the full space pool.

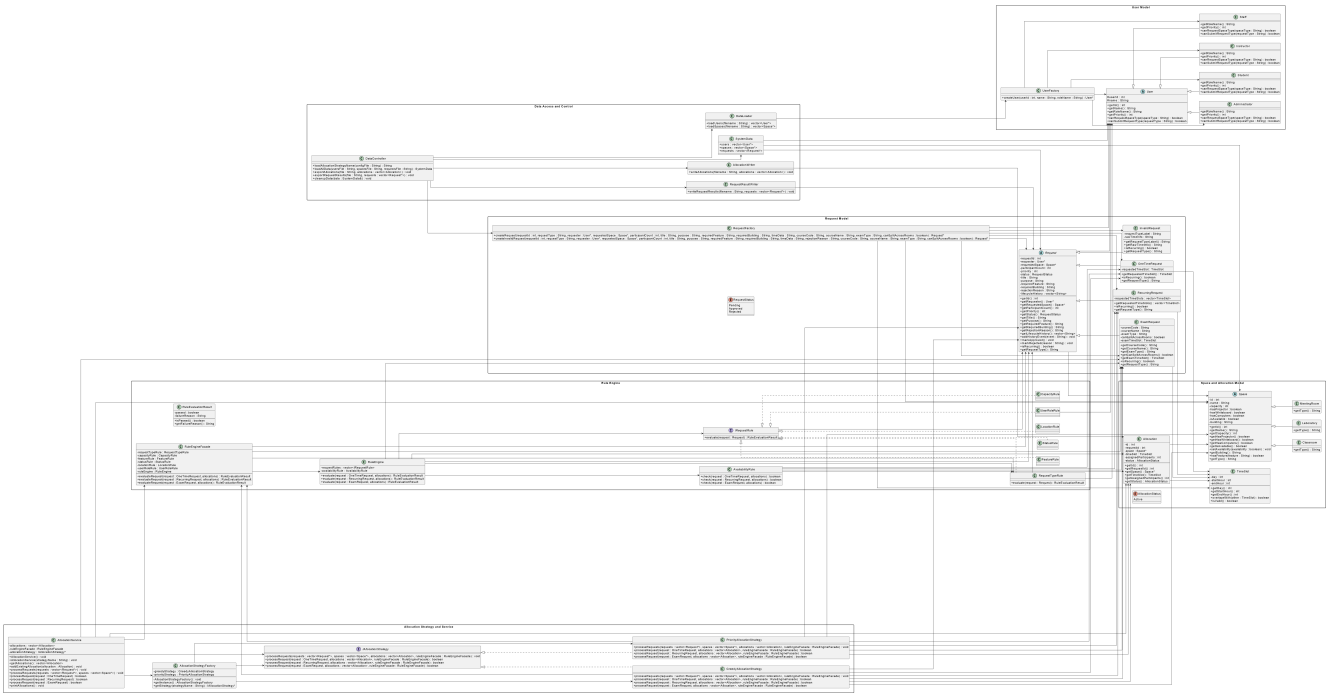

Figure 1: Updated Class Diagram with Assigned Participants and Space Pool Strategy Interface

8. Updated Workflow Diagram

Figure 2 shows the updated workflow. The program now passes both loaded requests and loaded spaces into `AllocationService`. The selected strategy receives the space pool, but current Greedy and Priority behavior remains unchanged. When an allocation is created, the assigned participant count is stored and later exported.

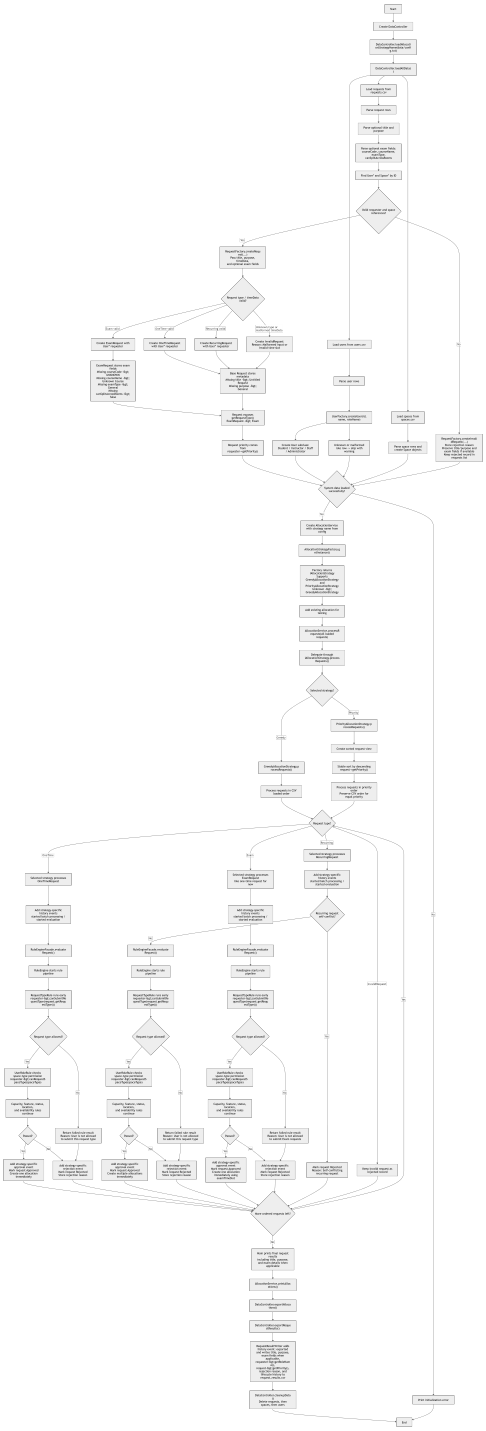

Figure 2: Workflow with Assigned Participants and Strategy Space Pool Access

9. Prototype Implementation

The C++ prototype was extended by updating the `Allocation` class. A new integer field, `assignedParticipants`, was added together with an accessor method. The allocation constructor was updated so every allocation explicitly receives this value.

All allocation creation paths were updated. One-time request allocations, recurring request occurrence allocations, and exam request allocations now pass `request.getParticipantCount()` as the assigned participant count. The existing manual allocation used for conflict testing was also updated with a realistic assigned participant value based on room capacity.

The allocation output path was updated as well. `AllocationService::printAllocations()` now displays assigned participants in the console, and `AllocationWriter` exports the new `assignedParticipants` column in `allocations.csv`. The request result export remains unchanged because request results already describe the total participant count at the request level.

The second improvement updates the allocation strategy interface. The batch method in `IAllocationStrategy` now receives the loaded space pool. `GreedyAllocationStrategy` and `PriorityAllocationStrategy` were updated to match the interface, but they intentionally do not use the space list yet. `AllocationService` now exposes a batch processing method that accepts both requests and spaces, and `main.cpp` passes `data.requests` and `data.spaces` into it.

10. Summary

This twenty-seventh implementation iteration prepares the allocation side of the system for future multi-room exam allocation. It adds assigned participant tracking to individual allocations and updates the strategy interface so future strategies can access the full pool of loaded spaces.

The current behavior remains unchanged. Greedy and Priority strategies still approve and reject the same requests, rules continue to validate requests in the same way, and no multi-room splitting or best-fit selection has been added yet. The system now has a clearer separation between total request demand and room-level assignment, which will support future exam allocation extensions.